



API Reference

Complete reference for all map components and their props.

Note: This library is built on top of [MapLibre GL JS](#). Most components extend the native MapLibre options. Refer to the [MapLibre Map API](#) for additional options not listed here.

Component Anatomy

All parts of the component that you can use and combine to build your map.

```
<Map>
  <MapMarker longitude={ ... } latitude={ ... }>
    <MarkerContent>
      <MarkerLabel />
    </MarkerContent>
    <MarkerPopup />
    <MarkerTooltip />
  </MapMarker>

  <MapPopup longitude={ ... } latitude={ ... } />
  <MapControls />
  <MapRoute coordinates={ ... } />
  <MapClusterLayer data={ ... } />
</Map>
```

Map

The root container component that initializes MapLibre GL and provides context to child components. Automatically handles theme switching between light and dark modes.

Extends [MapOptions](#) from MapLibre GL (excluding `container` and `style`).

Prop	Type	Default	Description
children	ReactNode	–	Child components (markers, popups, controls, routes).
className	string	–	Additional CSS classes for the map container.
theme	"light" "dark"	–	Theme for the map. If not provided, automatically detects from document class or system preference.
styles	{ light?: string StyleSpecification; dark?: string StyleSpecification }	–	Custom map styles for light and dark themes. Overrides the default Carto base map tiles.
projection	ProjectionSpecification	–	Map projection type. Use { type: "globe" } for 3D globe view.
viewport	Partial<MapViewport>	–	Controlled viewport state. When used with onViewportChange, enables controlled mode. Can also be used alone for initial viewport.
onViewportChange	(viewport: MapViewport) ⇒ void	–	Callback fired continuously as the viewport changes (during pan, zoom, rotate). Can be used alone to observe changes, or with viewport prop to enable controlled mode.

useMap

A hook that provides access to the MapLibre map instance and loading state. Must be used within a `Map` component.

```
const { map, isLoading } = useMap();
```

Returns `map` ([MapLibre.Map](#)) and `isLoading` (boolean) tells you if the map is loaded and ready to use.

MapControls

Renders map control buttons (zoom, compass, locate, fullscreen). Must be used inside `Map`.

Prop	Type	Default	Description
<code>position</code>	<code>"top-left" "top-right" "bottom-left" "bottom-right"</code>	<code>"bottom-right"</code>	Position of the controls on the map.
<code>showZoom</code>	<code>boolean</code>	<code>true</code>	Show zoom in/out buttons.
<code>showCompass</code>	<code>boolean</code>	<code>false</code>	Show compass button to reset bearing.
<code>showLocate</code>	<code>boolean</code>	<code>false</code>	Show locate button to find user's location.
<code>showFullscreen</code>	<code>boolean</code>	<code>false</code>	Show fullscreen toggle button.
<code>className</code>	<code>string</code>	<code>-</code>	Additional CSS classes for the controls container.
<code>onLocate</code>	<code>(coords: { longitude: number; latitude: number }) => void</code>	<code>-</code>	Callback with user coordinates when located.

MapMarker

A container for marker-related components. Provides context for its children and handles marker positioning.

Extends MarkerOptions from MapLibre GL (excluding `element`).

Prop	Type	Default	Description
<code>longitude</code>	<code>number</code>	–	Longitude coordinate for marker position.
<code>latitude</code>	<code>number</code>	–	Latitude coordinate for marker position.
<code>children</code>	<code>ReactNode</code>	–	Marker subcomponents (MarkerContent, MarkerPopup, etc).
<code>onClick</code>	<code>(e: MouseEvent) => void</code>	–	Callback when marker is clicked.
<code>onMouseEnter</code>	<code>(e: MouseEvent) => void</code>	–	Callback when mouse enters marker.
<code>onMouseLeave</code>	<code>(e: MouseEvent) => void</code>	–	Callback when mouse leaves marker.
<code>onDragStart</code>	<code>(lngLat: {lng, lat}) => void</code>	–	Callback when marker drag starts (requires draggable: true).
<code>onDrag</code>	<code>(lngLat: {lng, lat}) => void</code>	–	Callback during marker drag (requires draggable: true).
<code>onDragEnd</code>	<code>(lngLat: {lng, lat}) => void</code>	–	Callback when marker drag ends (requires draggable: true).

MarkerContent

Renders the visual content of a marker. Must be used inside `MapMarker` . If no children provided, renders a default blue dot marker.

Prop	Type	Default	Description
<code>children</code>	<code>ReactNode</code>	–	Custom marker content. Defaults to a blue dot.
<code>className</code>	<code>string</code>	–	Additional CSS classes for the marker container.

MarkerPopup

Renders a popup attached to the marker that opens on click. Must be used inside `MapMarker`.

Extends **PopupOptions** from MapLibre GL (excluding `className` and `closeButton`).

The `className` and `closeButton` from MapLibre's `PopupOptions` are excluded to prevent style conflicts. Use the component's own props to style the popup. MapLibre's default popup styles are reset via CSS.

Prop	Type	Default	Description
<code>children</code>	<code>ReactNode</code>	–	Popup content.
<code>className</code>	<code>string</code>	–	Additional CSS classes for the popup container.
<code>closeButton</code>	<code>boolean</code>	<code>false</code>	Show a close button in the popup.

MarkerTooltip

Renders a tooltip that appears on hover. Must be used inside `MapMarker`.

Extends **PopupOptions** from MapLibre GL (excluding `className`, `closeButton`, and `closeOnClick` as tooltips auto-dismiss on hover out).

The `className` from MapLibre's `PopupOptions` is excluded to prevent style conflicts. Use the component's own `className` prop to style the tooltip content. MapLibre's default popup styles are reset via CSS.

Prop	Type	Default	Description
children	ReactNode	–	Tooltip content.
className	string	–	Additional CSS classes for the tooltip container.

MarkerLabel

Renders a text label above or below the marker. Must be used inside `MarkerContent` .

Prop	Type	Default	Description
children	ReactNode	–	Label text content.
className	string	–	Additional CSS classes for the label.
position	"top" "bottom"	"top"	Position of the label relative to the marker.

MapPopup

A standalone popup component that can be placed anywhere on the map without a marker. Must be used inside `Map` .

Extends [PopupOptions](#) from MapLibre GL (excluding `className` and `closeButton`).

The `className` and `closeButton` from MapLibre's `PopupOptions` are excluded to prevent style conflicts. Use the component's own props to style the popup. MapLibre's default popup styles are reset via CSS.

Prop	Type	Default	Description
longitude	number	–	Longitude coordinate for popup position.
latitude	number	–	Latitude coordinate for popup position.

Prop	Type	Default	Description
onClose	<code>() ⇒ void</code>	–	Callback when popup is closed.
children	ReactNode	–	Popup content.
className	string	–	Additional CSS classes for the popup container.
closeButton	boolean	false	Show a close button in the popup.

MapRoute

Renders a line/route on the map connecting coordinate points. Must be used inside `Map`. Supports click and hover interactions for building route selection UIs.

Prop	Type	Default	Description
id	string	undefined (auto-generated)	Optional unique identifier for the route layer. Auto-generated if not provided.
coordinates	<code>[number, number][]</code>	–	Array of [longitude, latitude] coordinate pairs.
color	string	"#4285F4"	Line color (CSS color value).
width	number	3	Line width in pixels.
opacity	number	0.8	Line opacity (0 to 1).
dashArray	<code>[number, number]</code>	–	Dash pattern [dash length, gap length] for dashed lines.
onClick	<code>() ⇒ void</code>	–	Callback when the route line is clicked.
onMouseEnter	<code>() ⇒ void</code>	–	Callback when mouse enters the route line.

Prop	Type	Default	Description
<code>onMouseLeave</code>	<code>() => void</code>	–	Callback when mouse leaves the route line.
<code>interactive</code>	<code>boolean</code>	<code>true</code>	Whether the route is interactive (shows pointer cursor on hover).

MapClusterLayer

Renders clustered point data using MapLibre GL's native clustering. Automatically groups nearby points into clusters that expand on click. Must be used inside `Map`. Supports a generic type parameter for typed feature properties: `MapClusterLayer<MyProperties>`.

Prop	Type	Default	Description
<code>data</code>	<code>string GeoJSON.FeatureCollection</code>	–	GeoJSON FeatureCollection data or URL to fetch GeoJSON from.
<code>clusterMaxZoom</code>	<code>number</code>	<code>14</code>	Maximum zoom level to cluster points on.
<code>clusterRadius</code>	<code>number</code>	<code>50</code>	Radius of each cluster when clustering points (in pixels).
<code>clusterColors</code>	<code>[string, string, string]</code>	<code>["#22c55e", "#eab308", "#ef4444"]</code>	Colors for cluster circles: [small, medium, large] based on point count.
<code>clusterThresholds</code>	<code>[number, number]</code>	<code>[100, 750]</code>	Point count thresholds for color/size steps: [medium, large].

Prop	Type	Default	Description
pointColor	string	"#3b82f6"	Color for unclustered individual points.
onPointClick	(feature: GeoJSON.Feature, coordinates: [number, number]) => void	–	Callback when an unclustered point is clicked.
onClusterClick	(clusterId: number, coordinates: [number, number], pointCount: number) => void	–	Callback when a cluster is clicked. If not provided, zooms into the cluster.

[< Installation](#)

[Map >](#)